Lab Report: Advanced Observability and Distributed Tracing

OpenTelemetry instrumentation, Jaeger tracing and trace-correlated logging for a containerised Flask service on AWS

Prepared by: Celse Mizeromahire

Overview

A Flask weather service already monitored with Prometheus and Grafana was extended with distributed tracing and trace-correlated logging. Metrics could say that the error rate had risen; they could not say which requests failed or why.

The result is a single investigative path. An alert names a symptom, Jaeger identifies the requests behind it, and the trace ID carried in every log line retrieves that request's own output from CloudWatch.

How the signals flow

Instrumentation lives with the application, because it ships in the same image through the same pipeline. The tracing backend, dashboards and alert rules live with the observability platform, which is deployed separately and serves more than one application.

Prometheus scrapes application metrics every fifteen seconds. The application exports spans to Jaeger over OTLP and writes structured logs to CloudWatch through the Docker log driver. Prometheus evaluates the alert rules and forwards firing alerts to Alertmanager, which routes them to Discord. The three signals answer different questions, and the trace ID is what joins them: it is written into each log line at the moment the line is emitted, so any trace can be matched to its own output.

What was built

Capability	Component	Configuration
Tracing	OpenTelemetry SDK 1.29	A server span per request and a client span per outbound call, so time waiting on the upstream API is separated from time inside the handler. Exported over OTLP in batches on a background thread, adding no round trip to the request path.
Trace storage	Jaeger 2.20	An additional container on the existing monitoring host, configured by one YAML file in the OpenTelemetry Collector format. Spans held in Badger, an embedded key-value store, with 72-hour retention on local disk.
Metrics	Prometheus Flask exporter	RED metrics retained unchanged: rate and errors from a status-labelled request counter, duration from a histogram. Endpoint restricted to the VPC CIDR.
Log correlation	JSON formatter	Reads the active span context on every record and attaches the trace and span IDs as lowercase hexadecimal, matching how Jaeger displays them. One JSON line per request, collected by the Docker awslogs driver.
Dashboards	Grafana 11.5	Three rows: application error rate, throughput and p95 latency; host CPU, memory and disk; and a Traces row listing the slowest traces and those marked as errors, queried directly from Jaeger.
Alerting	2 rules, Alertmanager 0.28	5xx above five per cent of traffic, and p95 latency above 300ms, each held for ten minutes. Notifications carry the rule, value and instance, with links into Jaeger, the dashboard and

the alerts page.

Access to Jaeger is split by audience. The interface is reachable only from the operator's IP address, because a person uses it; the span ingest port only from inside the VPC, because services use it and an open ingest port would let anyone inject fabricated traces. Jaeger publishes its own metrics in Prometheus format and is scraped alongside everything else.

From symptom to root cause

Tracing initialises only when the OTLP endpoint variable is set, so local runs and tests are independent of the backend. Two fault paths, served by endpoints that exist only when a test route flag is set, were used to validate the chain end to end.

Errors. Sustained requests to an endpoint returning HTTP 500 raised the error rate above five per cent. The rule fired after ten minutes and the notification reached Discord. Its Jaeger link returned the traces carrying an error tag; opening one placed the failure inside the Flask server span rather than in the outbound API call, which distinguishes a fault in our own code from a fault in an upstream dependency. Filtering CloudWatch on that trace ID returned the log lines for that single request.

Latency. Sustained requests to an endpoint delaying 800 milliseconds raised p95 above the threshold. The same sequence followed, with the trace placing the delay inside the handler rather than in the upstream call.

The evidence screenshots show the same trace ID in Jaeger and in CloudWatch, which is what demonstrates that the three signals describe one system rather than three.

Verification

Signal	Result
Metrics	Seven Prometheus targets healthy, including Jaeger's own; the metrics endpoint serves RED metrics and counts 5xx responses.
Traces	The application appears in Jaeger with server and client spans on every trace, and the Grafana Traces row populates from the same data.
Logs and alerting	CloudWatch holds JSON lines carrying trace and span IDs; both alert rules load, evaluate, were observed firing under induced load, and cleared automatically on recovery.

Conclusion

The stack meets each stated requirement: OpenTelemetry instrumentation of the HTTP server and client exporting to Jaeger, RED metrics on the metrics endpoint, structured JSON logs carrying trace and span IDs into CloudWatch, a Grafana dashboard covering latency, error rate, host resources and trace links, and alert rules at five per cent errors and 300ms latency over ten minutes. The correlation from alert to trace to log was validated on both an error path and a latency path, and instrumentation, dashboards and alert rules are all defined in code.

Links

Application and instrumentation: github.com/Smiley2507/jenkins-cicd-lab
Observability platform: github.com/Smiley2507/monitoring-and-security-lab